

HerAI

LARGE LANGUAGE MODEL

Modul Praktis dan Mudah Dipahami

Dari tokenisasi dan Transformer sampai RAG, evaluasi, safety, dan deployment

Program	HerAI Fellowship
Kategori	Foundation and Core AI
Level	Intermediate
Durasi	20-24 jam
Versi	1.0.0 - 23 Juli 2026

Participant Module

Belajar konsep, membangun sistem, mengukur kualitas, dan mengelola risiko

Tentang Modul

Modul ini dirancang sebagai bahan belajar peserta HerAI Fellowship. Bahasa, contoh, latihan, dan project disusun agar konsep teknis dapat dipahami secara bertahap tanpa menghilangkan prinsip rekayasa dan tata kelola yang penting.

Gunakan modul dengan pola baca - praktik - evaluasi. Setelah membaca satu bab, kerjakan checkpoint tanpa melihat kembali materi. Kemudian lakukan latihan dan catat bagian yang masih membingungkan.

Contoh kode dalam modul bertujuan menjelaskan konsep. Sebelum dipakai pada sistem nyata, tambahkan error handling, autentikasi, authorization, logging yang aman, pengujian, dan review keamanan.

Nama model atau penyedia tidak dijadikan pusat materi. Peserta diharapkan dapat menerapkan prinsip pada model dan platform yang berbeda.

Cara Menggunakan Modul

- Baca tujuan bab sebelum memulai.
- Tuliskan ulang konsep dengan bahasa sendiri.
- Kerjakan checkpoint dan latihan.
- Jalankan contoh kode secara lokal bila memungkinkan.
- Simpan hasil eksperimen dan alasan perubahan.
- Gunakan eval set yang sama ketika membandingkan versi.

Peta Belajar

```
Fondasi bahasa dan Transformer
↓
Cara model dilatih dan menghasilkan teks
↓
Prompting dan keluaran terstruktur
↓
RAG dan fine-tuning
↓
Evaluasi, safety, privacy, dan security
↓
Deployment dan mini project
```

Daftar Isi

- 01** Bab 1 - Pengantar Large Language Model
- 02** Bab 2 - Tokenisasi dan Representasi Teks
- 03** Bab 3 - Embedding dan Informasi Posisi
- 04** Bab 4 - Transformer untuk Model Bahasa
- 05** Bab 5 - Pretraining, Instruction Tuning, dan Alignment
- 06** Bab 6 - Inference dan Strategi Decoding
- 07** Bab 7 - Prompting yang Terstruktur
- 08** Bab 8 - Retrieval-Augmented Generation
- 09** Bab 9 - Fine-Tuning dan Adaptasi Model
- 10** Bab 10 - Evaluasi Sistem LLM
- 11** Bab 11 - Hallucination, Safety, Privacy, dan Security
- 12** Bab 12 - Deployment, Monitoring, Latency, dan Cost
- 13** Bab 13 - Mini Project Asisten FAQ Berbasis Dokumen
- 14** Bab 14 - Kuis Akhir dan Pembahasan
- 15** Bab 15 - Diskusi, Glosarium, Ringkasan, dan Referensi

Ikhtisar Course

Modul belajar praktis dan mudah dipahami untuk peserta HerAI Fellowship. Materi dimulai dari cara teks diubah menjadi token, kemudian membahas embedding, Transformer, pretraining, prompting, Retrieval-Augmented Generation, fine-tuning, evaluasi, keselamatan, deployment, dan mini project asisten FAQ berbasis dokumen.

Informasi Course

Komponen	Keterangan
Level	Intermediate
Estimasi belajar	20-24 jam
Prasyarat	Python dasar, machine learning, pengantar deep learning, dan konsep Transformer dasar
Bentuk pembelajaran	Materi, analogi, diagram teks, contoh, latihan, checkpoint, kuis, diskusi, dan mini project
Hasil akhir	Peserta mampu memahami alur kerja LLM dan merancang prototipe aplikasi LLM yang terukur, aman, dan dapat dievaluasi

Tentang Modul Ini

Large Language Model atau LLM sering terlihat seperti sistem yang memahami bahasa sebagaimana manusia. Dalam praktiknya, LLM adalah model statistik berskala besar yang mempelajari pola token dari banyak data dan menghasilkan token berikutnya berdasarkan konteks yang tersedia.

Modul ini tidak hanya menjelaskan cara meminta jawaban dari model. Peserta akan belajar membedakan kemampuan model, pengetahuan eksternal, instruksi, konteks, dan komponen sistem di sekeliling model. Pemahaman tersebut penting agar aplikasi tidak dibangun berdasarkan asumsi bahwa model selalu benar.

Prinsip belajar: pahami dulu aliran data dan sumber kegagalan. Baru setelah itu pilih teknik seperti prompting, RAG, atau fine-tuning. Teknik yang lebih kompleks tidak otomatis menjadi solusi yang lebih baik.

Capaian Pembelajaran

Setelah menyelesaikan modul ini, peserta diharapkan mampu:

1. Menjelaskan LLM sebagai model prediksi token dan membedakannya dari mesin pencari atau basis data.
2. Memahami tokenisasi, vocabulary, embedding, positional information, attention, dan Transformer decoder secara intuitif.
3. Menjelaskan pretraining, instruction tuning, preference alignment, dan proses inference.
4. Mengendalikan keluaran melalui instruksi, konteks, contoh, format, dan parameter decoding.
5. Menentukan kapan menggunakan prompting, RAG, fine-tuning, atau kombinasi beberapa pendekatan.
6. Mendesain pipeline RAG sederhana yang memiliki proses chunking, retrieval, generation, dan citation.
7. Mengevaluasi kualitas output berdasarkan correctness, relevance, faithfulness, safety, latency, dan cost.
8. Mengenali hallucination, prompt injection, kebocoran data, bias, serta risiko operasional lainnya.
9. Membuat mini project asisten FAQ berbasis dokumen dan menyusun laporan eksperimen.

Prasyarat

Peserta sebaiknya sudah memahami variabel, fungsi, list, dictionary, dan pemrosesan string di Python. Pemahaman dasar tentang neural network, loss function, gradient descent, embedding, dan attention akan membantu, tetapi setiap konsep utama tetap dijelaskan secara bertahap.

Catatan matematika: modul memakai notasi sederhana untuk menunjukkan hubungan antarkomponen. Peserta tidak diwajibkan menurunkan seluruh persamaan Transformer.

Peta Materi

1. Pengantar Large Language Model
2. Tokenisasi dan Representasi Teks
3. Embedding dan Informasi Posisi
4. Transformer untuk Model Bahasa
5. Pretraining, Instruction Tuning, dan Alignment
6. Inference dan Strategi Decoding
7. Prompting yang Terstruktur
8. Retrieval-Augmented Generation
9. Fine-Tuning dan Adaptasi Model
10. Evaluasi Sistem LLM
11. Hallucination, Safety, Privacy, dan Security
12. Deployment, Monitoring, Latency, dan Cost
13. Mini Project Asisten FAQ Berbasis Dokumen
14. Kuis Akhir dan Pembahasan
15. Diskusi, Glosarium, Ringkasan, dan Referensi

Bab 1 - Pengantar Large Language Model

1.1 Apa Itu Large Language Model?

Large Language Model adalah model yang mempelajari pola bahasa dari kumpulan teks dalam skala besar. Kata *large* dapat merujuk pada jumlah parameter, jumlah data, kebutuhan komputasi, atau gabungan ketiganya. Kata *language model* berarti model memperkirakan kemungkinan urutan token.

Untuk model autoregresif, gagasan dasarnya adalah:

```
teks sebelumnya -> model -> probabilitas token berikutnya
```

Misalnya, setelah membaca kalimat Ibu membeli beras di, model mungkin memberi kemungkinan tinggi pada token pasar, toko, atau warung. Model memilih atau mengambil sampel satu token, menambahkannya ke konteks, lalu mengulangi proses tersebut sampai respons selesai.

Secara ringkas:

```
Prompt: "Air membeku pada suhu"
↓
Model menghitung distribusi probabilitas token
↓
Token terpilih: "0"
↓
Konteks baru: "Air membeku pada suhu 0"
↓
Prediksi berikutnya: "derajat", lalu "Celsius"
```

1.2 LLM Bukan Mesin Pencari

LLM dan mesin pencari memiliki cara kerja berbeda.

Aspek	LLM	Mesin Pencari
Fungsi utama	Menghasilkan teks berdasarkan pola dan konteks	Menemukan dokumen atau halaman yang relevan
Sumber jawaban	Parameter model dan konteks yang diberikan	Indeks dokumen yang dapat ditelusuri
Jaminan sumber	Tidak otomatis memiliki sumber yang dapat diperiksa	Biasanya menampilkan tautan atau dokumen
Risiko utama	Menghasilkan jawaban terdengar benar tetapi salah	Menemukan sumber yang tidak relevan atau kurang tepercaya
Kekuatan	Merangkum, mengubah format, menulis, menjelaskan	Menemukan informasi aktual dan sumber asli

Aplikasi yang membutuhkan fakta dari dokumen tertentu sebaiknya tidak hanya mengandalkan memori parameter model. Sistem dapat menambahkan retrieval sehingga model menerima potongan dokumen yang relevan sebelum menjawab.

1.3 Kemampuan yang Sering Muncul

LLM dapat menunjukkan kemampuan seperti:

- menjawab pertanyaan,
- merangkum teks,
- menerjemahkan,
- mengklasifikasikan teks,
- mengekstrak informasi,

- menghasilkan kode,
- mengubah gaya penulisan,
- mengikuti format keluaran,
- menggunakan dokumen atau tool yang diberikan sistem.

Kemampuan tersebut tidak berarti model memiliki pemahaman, ingatan, atau tujuan yang sama dengan manusia. Output tetap dipengaruhi pola data, instruksi, konteks, dan proses sampling.

1.4 Kapan LLM Cocok Digunakan?

LLM cocok ketika tugas melibatkan bahasa yang bervariasi dan aturan manual sulit mencakup seluruh kemungkinan, misalnya:

- membuat ringkasan awal dari dokumen panjang,
- menyusun draf komunikasi yang kemudian ditinjau manusia,
- mengubah pertanyaan pengguna menjadi kategori atau parameter terstruktur,
- menjawab FAQ berdasarkan basis pengetahuan,
- membantu pencarian semantik,
- menghasilkan variasi materi belajar.

LLM kurang tepat ketika:

- jawaban harus selalu deterministik dan dapat dihitung dengan aturan sederhana,
- kesalahan kecil dapat menimbulkan dampak keselamatan tinggi tanpa verifikasi,
- sumber data tidak boleh meninggalkan lingkungan yang dikendalikan,
- informasi harus selalu mutakhir tetapi sistem tidak memiliki retrieval,
- tugas cukup diselesaikan dengan template, regex, query database, atau model kecil.

Prinsip praktis: gunakan LLM untuk bagian yang membutuhkan fleksibilitas bahasa. Gunakan program biasa untuk aturan pasti, perhitungan, validasi, dan kontrol akses.

Checkpoint Bab 1

1. Mengapa LLM disebut model prediksi token?
2. Apa perbedaan paling penting antara LLM dan mesin pencari?
3. Sebutkan satu kasus ketika aturan biasa lebih tepat daripada LLM.

Latihan Bab 1 - Menentukan Peran LLM

Pilih salah satu skenario berikut: layanan pelanggan, tutor belajar, administrasi kampus, atau analisis dokumen. Buat tabel berisi:

- tugas yang cocok ditangani LLM,
- tugas yang harus ditangani program deterministik,
- bagian yang harus ditinjau manusia,
- risiko jika seluruh proses diserahkan kepada model.

Bab 2 - Tokenisasi dan Representasi Teks

2.1 Komputer Tidak Membaca Kata Secara Langsung

Model menerima angka, bukan kata. Sebelum teks masuk ke neural network, tokenizer memecah teks menjadi unit yang disebut **token**, kemudian setiap token dipetakan ke sebuah ID integer.

```
"belajar AI itu menarik"
  ↓ tokenisasi
["belajar", " AI", " itu", " menarik"]
  ↓ token ID
[1842, 927, 311, 6021]
```

Contoh di atas hanya ilustrasi. Token sebenarnya bergantung pada tokenizer dan vocabulary model. Satu kata dapat menjadi satu token, beberapa token, atau bergabung dengan spasi dan tanda baca.

2.2 Mengapa Tidak Selalu Per Kata?

Tokenisasi per kata memiliki masalah:

- vocabulary menjadi sangat besar,
- kata baru atau salah ketik sulit ditangani,
- bahasa dengan banyak imbuhan menghasilkan banyak variasi,
- nama, kode, dan istilah teknis sering tidak tersedia.

Tokenizer modern sering memakai unit subword. Kata *ketidakpastian* dapat dipecah menjadi bagian yang lebih kecil. Dengan cara ini, model dapat menangani kata yang jarang muncul menggunakan potongan yang sudah dikenal.

Pendekatan	Contoh unit	Kelebihan	Keterbatasan
Karakter	b, e, l, a, j, a, r	Vocabulary kecil	Urutan menjadi panjang
Kata	belajar	Mudah dipahami	Sulit menangani kata baru
Subword	bel, ajar	Seimbang dan fleksibel	Pemotongan tidak selalu intuitif
Byte	Nilai byte	Dapat mewakili semua teks	Urutan dapat lebih panjang

2.3 Vocabulary dan Token ID

Vocabulary adalah daftar token yang dikenali tokenizer. Setiap token memiliki ID. ID tidak menunjukkan makna atau urutan kualitas. Token ID hanyalah alamat yang dipakai untuk mengambil embedding.

```
# Ilustrasi sederhana, bukan tokenizer produksi.
vocabulary = {
    "<unk>": 0,
    "belajar": 1,
    "AI": 2,
    "menarik": 3,
}

sentence = ["belajar", "AI", "menarik"]
token_ids = [vocabulary.get(word, 0) for word in sentence]
print(token_ids) # [1, 2, 3]
```

Tokenizer produksi memiliki aturan normalisasi, pemisahan, penggabungan subword, token khusus, dan decoding kembali ke teks.

2.4 Token Khusus

Model dapat memakai token khusus untuk menandai struktur, misalnya:

- awal atau akhir teks,
- pemisah antarbagian,
- padding agar panjang batch seragam,
- peran system, user, dan assistant,
- token untuk tool atau format tertentu.

Prompt yang terlihat sama bagi pengguna dapat menghasilkan token yang berbeda jika template percakapannya berbeda.

2.5 Dampak Tokenisasi

Tokenisasi memengaruhi:

1. **Panjang konteks.** Batas model dihitung dalam token, bukan jumlah kata.
2. **Biaya dan latency.** Lebih banyak token berarti lebih banyak komputasi.
3. **Kemampuan multibahasa.** Bahasa yang terwakili buruk dapat membutuhkan lebih banyak token.
4. **Pemrosesan kode dan angka.** Urutan simbol dapat dipecah dengan cara yang berbeda.
5. **Pemotongan dokumen.** Chunk sebaiknya diukur berdasarkan token atau estimasi yang cukup dekat.

Kesalahan umum: menganggap 1 token selalu sama dengan 1 kata. Hubungannya berubah menurut bahasa, tanda baca, dan tokenizer.

Checkpoint Bab 2

1. Mengapa model perlu mengubah teks menjadi token ID?
2. Apa keuntungan tokenisasi subword?
3. Bagaimana tokenisasi dapat memengaruhi biaya aplikasi?

Latihan Bab 2 - Audit Tokenisasi Konseptual

Ambil lima contoh teks:

- kalimat bahasa Indonesia formal,
- kalimat dengan slang,
- alamat web,
- potongan kode,
- kalimat dengan angka dan satuan.

Perkirakan bagian mana yang mungkin dipecah menjadi banyak token. Jelaskan dampaknya terhadap context window dan chunking.

Bab 3 - Embedding dan Informasi Posisi

3.1 Dari Token ID ke Vektor

Token ID belum membawa informasi yang dapat diproses secara bermakna. Model memakai tabel embedding untuk mengubah ID menjadi vektor berisi angka.

```
Token ID 1842
  ↓ lookup embedding
[0.12, -0.43, 0.08, ..., 0.31]
```

Vektor embedding dipelajari selama training. Token yang sering muncul dalam konteks serupa dapat memiliki representasi yang membantu model mengolah hubungan semantik dan sintaksis.

Analogi: token ID seperti nomor anggota perpustakaan. Nomor tersebut tidak menjelaskan minat pembaca. Embedding seperti profil multidimensi yang merangkum pola keterkaitan.

3.2 Embedding Tidak Sama dengan Definisi Kamus

Embedding bukan daftar makna eksplisit. Setiap dimensi biasanya tidak memiliki label sederhana seperti `tingkat formalitas` atau `jenis objek`. Makna muncul dari pola gabungan banyak dimensi dan berubah ketika embedding diproses oleh layer Transformer.

Pada awal model, setiap kemunculan token yang sama mengambil embedding awal yang sama. Setelah melewati self-attention dan feed-forward layer, representasinya menjadi **contextual**. Token `bisa` pada kalimat `ular itu berbisa` dan `saya bisa hadir` akhirnya memiliki representasi berbeda karena konteksnya berbeda.

3.3 Mengapa Posisi Dibutuhkan?

Self-attention membandingkan token satu sama lain. Tanpa informasi posisi, model sulit membedakan:

- kucing mengejar tikus, dan
- tikus mengejar kucing.

Karena itu, model menambahkan atau menggabungkan informasi posisi dengan representasi token. Pendekatan dapat berupa positional encoding tetap, embedding posisi yang dipelajari, atau mekanisme posisi relatif.

```
representasi input = token embedding + informasi posisi
```

3.4 Similarity untuk Retrieval

Embedding juga digunakan di luar model generatif. Sebuah embedding model dapat mengubah kalimat atau dokumen menjadi vektor. Kedekatan dua vektor dapat dipakai untuk pencarian semantik.

Cosine similarity secara intuitif membandingkan arah dua vektor:

```
similarity(A, B) = dot(A, B) / (panjang(A) × panjang(B))
```

Nilai yang lebih tinggi biasanya menunjukkan representasi lebih mirip menurut model embedding. Similarity bukan bukti bahwa dua teks benar, setara, atau aman. Ia hanya sinyal kedekatan yang perlu dievaluasi.

3.5 Dense dan Sparse Retrieval

Pendekatan	Representasi	Kekuatan	Keterbatasan
Sparse retrieval	Kata dan bobot kemunculan	Kuat untuk istilah exact, nama, kode	Kurang fleksibel pada parafrasa
Dense retrieval	Vektor embedding	Kuat untuk kemiripan semantik	Dapat melewatkan istilah exact
Hybrid retrieval	Gabungan keduanya	Menyeimbangkan exact dan semantik	Pipeline lebih kompleks

Checkpoint Bab 3

1. Apa fungsi embedding pada input LLM?
2. Mengapa token yang sama dapat memiliki makna kontekstual berbeda?
3. Kapan sparse retrieval dapat lebih berguna daripada dense retrieval?

Latihan Bab 3 - Membandingkan Makna

Buat enam pasangan kalimat. Tiga pasangan harus memiliki kata berbeda tetapi makna mirip. Tiga pasangan lainnya harus memiliki banyak kata sama tetapi makna berbeda. Jelaskan pendekatan retrieval mana yang mungkin berhasil atau gagal pada setiap pasangan.

Bab 4 - Transformer untuk Model Bahasa

4.1 Gambaran Besar

Transformer adalah arsitektur neural network yang mengolah hubungan antartoken menggunakan attention. Banyak LLM generatif memakai susunan **decoder-only Transformer**.

Alur sederhananya:

```

Token IDs
↓
Token embedding + posisi
↓
Transformer block 1
↓
Transformer block 2
↓
...
↓
Transformer block N
↓
Logits untuk seluruh vocabulary
↓
Probabilitas token berikutnya
  
```

Setiap Transformer block umumnya memiliki:

- self-attention,
- feed-forward network,
- residual connection,
- normalization.

4.2 Query, Key, dan Value

Self-attention membentuk tiga representasi dari setiap token:

- **Query:** informasi apa yang sedang dicari token ini?
- **Key:** informasi apa yang ditawarkan token lain?
- **Value:** informasi apa yang akan diteruskan jika dianggap relevan?

Analogi perpustakaan: query adalah pertanyaan pembaca, key adalah label katalog, dan value adalah isi buku yang dipinjam. Kecocokan query-key menentukan seberapa besar isi value digunakan.

Secara konseptual:

```

score = kecocokan(query, key)
weight = softmax(score)
output = jumlah(weight x value)
  
```

4.3 Causal Mask

Pada model autoregresif, token pada posisi tertentu tidak boleh melihat token masa depan saat training. Causal mask memastikan prediksi token ke-5 hanya menggunakan token 1 sampai 4.

Posisi yang boleh diperhatikan

```

Token 1: [1, 0, 0, 0]
Token 2: [1, 1, 0, 0]
Token 3: [1, 1, 1, 0]
Token 4: [1, 1, 1, 1]
  
```

Tanpa mask, model dapat melihat jawaban yang seharusnya diprediksi dan objective training menjadi tidak valid.

4.4 Multi-Head Attention

Satu pola attention mungkin fokus pada hubungan subjek-predikat, sedangkan pola lain fokus pada referensi kata, format, atau hubungan jarak jauh. Multi-head attention memberi beberapa ruang representasi yang dipelajari secara paralel.

Tidak ada jaminan setiap head memiliki fungsi manusiawi yang jelas. Namun secara kolektif, multi-head attention membantu model memproses berbagai hubungan.

4.5 Feed-Forward Network

Setelah informasi antartoken dicampur oleh attention, setiap posisi diproses oleh feed-forward network. Bagian ini memberi transformasi nonlinear dan kapasitas tambahan.

Attention: bertukar informasi antartoken
Feed-forward: memproses informasi pada setiap posisi

4.6 Residual dan Normalization

Residual connection membantu informasi dan gradient mengalir melewati banyak layer. Normalization menjaga skala aktivasi lebih stabil. Kedua komponen penting agar training model dalam dapat berjalan.

4.7 Dari Hidden State ke Logits

Representasi terakhir diproyeksikan menjadi satu skor untuk setiap token dalam vocabulary. Skor tersebut disebut **logits**. Softmax mengubah logits menjadi distribusi probabilitas.

```
import math

logits = [2.0, 1.0, 0.1]
exp_values = [math.exp(x) for x in logits]
total = sum(exp_values)
probabilities = [x / total for x in exp_values]
print(probabilities)
```

Token dengan probabilitas terbesar tidak selalu harus dipilih. Strategi decoding menentukan bagaimana probabilitas digunakan.

Checkpoint Bab 4

1. Apa tujuan self-attention?
2. Mengapa model autoregresif membutuhkan causal mask?
3. Apa perbedaan peran attention dan feed-forward network?

Latihan Bab 4 - Menelusuri Alur Data

Gambar ulang alur dari kalimat input sampai distribusi token berikutnya. Untuk setiap tahap, tuliskan bentuk informasi yang dihasilkan dan pertanyaan berikut:

- Apa yang dipelajari pada tahap tersebut?
- Apa yang terjadi jika tahap itu dihilangkan?
- Apakah tahap itu bekerja per token atau lintas token?

Bab 5 - Pretraining, Instruction Tuning, dan Alignment

5.1 Pretraining

Pada pretraining autoregresif, model berlatih memprediksi token berikutnya dari banyak urutan teks. Dataset dapat berisi berbagai jenis dokumen. Sebelum training, data perlu melalui filtering, deduplication, quality control, dan pengelolaan hak serta privasi.

Contoh satu data training:

```
Input : "Indonesia memiliki banyak"
Target: "pulau"
```

Dalam implementasi sebenarnya, banyak posisi pada urutan dapat berkontribusi pada loss secara bersamaan.

5.2 Cross-Entropy Loss

Model menghasilkan probabilitas untuk vocabulary. Cross-entropy memberi penalti ketika probabilitas token target rendah.

```
Target benar: "pulau"
Probabilitas model untuk "pulau": 0.70 -> loss relatif kecil
Probabilitas model untuk "pulau": 0.01 -> loss besar
```

Gradient descent memperbarui parameter agar prediksi pada data training membaik. Training skala besar membutuhkan data, komputasi, optimisasi, dan sistem terdistribusi yang matang.

5.3 Apa yang Dipelajari Saat Pretraining?

Model dapat mempelajari:

- pola tata bahasa,
- hubungan kata dan konteks,
- struktur dokumen,
- pola penalaran yang muncul dalam data,
- pengetahuan yang sering dinyatakan dalam teks,
- gaya dan bias dari data training.

Model tidak menyimpan database fakta dalam bentuk tabel yang mudah diperiksa. Pengetahuan terdistribusi di parameter dan dapat muncul secara tidak konsisten.

5.4 Instruction Tuning

Model pretrained mampu melanjutkan teks, tetapi belum tentu mengikuti permintaan pengguna dengan baik. Instruction tuning melatih model menggunakan pasangan instruksi dan respons yang diinginkan.

```
Instruksi: Ringkas paragraf berikut menjadi tiga poin.
Respons : Tiga poin ringkasan yang sesuai format.
```

Instruction tuning membantu model mengenali pola tugas, mengikuti format, dan merespons percakapan.

5.5 Preference Alignment

Preference alignment memakai perbandingan respons atau feedback untuk mendorong perilaku yang lebih membantu dan aman. Beberapa keluarga metode menggunakan reward model dan reinforcement learning; metode lain mengoptimalkan preferensi secara lebih langsung.

Tujuan alignment dapat meliputi:

- mengikuti instruksi,
- menghindari konten berbahaya,
- mengakui ketidakpastian,
- menjaga gaya dan format,
- memprioritaskan respons yang dinilai lebih membantu.

Alignment bukan jaminan model selalu aman atau benar. Evaluasi dan guardrail aplikasi tetap diperlukan.

5.6 Base Model dan Instruction Model

Jenis	Karakteristik	Penggunaan umum
Base model	Fokus melanjutkan teks	Riset, fine-tuning lanjutan, completion terkontrol
Instruction model	Dilatih mengikuti instruksi	Chatbot, asisten, ekstraksi, ringkasan
Domain-adapted model	Disesuaikan pada domain tertentu	Bahasa, hukum, kesehatan, kode, atau domain organisasi

5.7 Data Berkualitas Lebih Penting daripada Sekadar Banyak

Data duplikat, salah, berbahaya, atau tidak relevan dapat memperburuk model. Kualitas data mencakup:

- kesesuaian dengan tujuan,
- variasi yang cukup,
- ketepatan label atau respons,
- perlindungan data pribadi,
- dokumentasi sumber dan izin,
- keseimbangan kelompok dan bahasa,
- pemeriksaan kontaminasi evaluasi.

Checkpoint Bab 5

1. Apa tujuan pretraining autoregresif?
2. Mengapa pretrained model belum tentu menjadi chatbot yang baik?
3. Mengapa alignment tidak menggantikan evaluasi aplikasi?

Latihan Bab 5 - Audit Dataset Instruksi

Buat lima contoh instruksi dan respons untuk tugas ekstraksi informasi. Audit setiap contoh berdasarkan:

- kejelasan instruksi,
- konsistensi format,
- kebenaran respons,
- keberadaan data sensitif,
- kemungkinan bias,
- apakah contoh tersebut benar-benar mewakili penggunaan nyata.

Bab 6 - Inference dan Strategi Decoding

6.1 Apa Itu Inference?

Inference adalah proses menjalankan model yang sudah dilatih untuk menghasilkan output. Pada model autoregresif, inference berlangsung token demi token.

1. Model membaca prompt.
2. Model menghitung logits token berikutnya.
3. Decoding memilih token.
4. Token ditambahkan ke konteks.
5. Proses berulang sampai kondisi berhenti.

Dua fase komputasi sering dibedakan:

- **Prefill:** memproses seluruh token prompt.
- **Decode:** menghasilkan token baru satu per satu.

Prompt panjang meningkatkan beban prefill. Output panjang meningkatkan jumlah langkah decode.

6.2 Greedy Decoding

Greedy decoding selalu memilih token dengan probabilitas tertinggi.

Kelebihan:

- sederhana,
- lebih konsisten,
- cocok untuk beberapa tugas terstruktur.

Keterbatasan:

- dapat menghasilkan teks repetitif,
- tidak selalu menemukan urutan terbaik secara global,
- kurang bervariasi untuk tugas kreatif.

6.3 Temperature

Temperature mengubah ketajaman distribusi probabilitas sebelum sampling.

- Temperature rendah membuat pilihan lebih terkonsentrasi pada token berprobabilitas tinggi.
- Temperature lebih tinggi meningkatkan variasi.
- Temperature tidak membuat model lebih mengetahui fakta.

Tugas ekstraksi JSON -> biasanya butuh variasi rendah
Brainstorm ide kreatif -> dapat memakai variasi lebih tinggi

Nilai parameter bergantung pada model dan sistem. Selalu uji dengan dataset tugas nyata.

6.4 Top-k dan Top-p

Top-k sampling membatasi pilihan pada k token dengan probabilitas tertinggi.

Top-p sampling memilih himpunan token terkecil yang jumlah probabilitasnya mencapai ambang p.

Keduanya mengurangi kemungkinan memilih token dari ekor distribusi yang sangat kecil. Parameter dapat berinteraksi dengan temperature.

6.5 Stop Condition dan Maximum Output

Sistem perlu menentukan kapan generasi berhenti:

- token akhir dari model,
- stop sequence,

- batas token output,
- format selesai,
- pembatalan oleh pengguna atau sistem.

Batas output yang terlalu pendek memotong jawaban. Batas terlalu panjang meningkatkan biaya dan risiko respons menyimpang.

6.6 Repetition dan Structured Output

Beberapa sistem menyediakan penalti repetisi atau constraint format. Namun format tetap perlu divalidasi oleh program.

```
import json

raw_output = '{"category": "billing", "priority": 2}'

try:
    result = json.loads(raw_output)
    assert result["category"] in {"billing", "technical", "general"}
    assert result["priority"] in {1, 2, 3}
except (json.JSONDecodeError, KeyError, AssertionError):
    result = {"error": "Output model tidak valid"}
```

Prinsip penting: prompt meminta format. Program memastikan format.

6.7 Reproducibility

Output sampling dapat berubah antar-run. Bahkan konfigurasi yang terlihat sama dapat berubah karena versi model, sistem inference, atau implementasi numerik. Untuk eksperimen:

- simpan model dan versi,
- simpan prompt lengkap,
- simpan parameter decoding,
- gunakan dataset evaluasi tetap,
- catat waktu dan lingkungan,
- jangan menilai kualitas dari satu contoh saja.

Checkpoint Bab 6

1. Apa perbedaan fase prefill dan decode?
2. Mengapa temperature tinggi tidak berarti jawaban lebih benar?
3. Mengapa output JSON tetap harus divalidasi program?

Latihan Bab 6 - Eksperimen Decoding

Gunakan satu tugas ringkasan dan satu tugas brainstorming. Buat rancangan eksperimen yang membandingkan tiga konfigurasi decoding. Tentukan:

- parameter yang diubah,
- output yang diamati,
- rubric kualitas,
- jumlah pengulangan,
- cara memilih konfigurasi akhir.

Bab 7 - Prompting yang Terstruktur

7.1 Prompt Adalah Spesifikasi Tugas

Prompt yang baik menjelaskan pekerjaan yang harus dilakukan, bukan sekadar topik. Struktur yang berguna:

1. Peran atau konteks sistem.
2. Tujuan tugas.
3. Data atau konteks yang boleh digunakan.
4. Batasan.
5. Format keluaran.
6. Contoh bila diperlukan.
7. Kriteria keberhasilan.

Contoh prompt lemah:

```
Jelaskan laporan ini.
```

Contoh lebih terstruktur:

```
Tugas: ringkas laporan untuk koordinator divisi.
Gunakan hanya informasi dalam <dokumen>.
Keluaran:
1. tiga temuan utama,
2. dua risiko,
3. tindakan yang perlu diputuskan.
Jika informasi tidak tersedia, tulis "tidak ditemukan".
Maksimum 250 kata.
```

7.2 Memisahkan Instruksi dan Data

Gunakan delimiter agar model dapat membedakan instruksi dengan data pengguna.

```
<instruksi>
Ekstrak nama kegiatan, tanggal, dan penanggung jawab.
</instruksi>

<dokumen>
...
</dokumen>
```

Delimiter tidak menjadi sistem keamanan yang sempurna, tetapi meningkatkan kejelasan. Data eksternal tetap harus dianggap tidak tepercaya.

7.3 Zero-Shot, One-Shot, dan Few-Shot

Teknik	Isi prompt	Kapan berguna
Zero-shot	Instruksi tanpa contoh	Tugas sederhana atau model sudah mengenali pola
One-shot	Satu contoh	Menunjukkan format dasar
Few-shot	Beberapa contoh	Menstabilkan label, gaya, atau edge case

Contoh few-shot harus benar, konsisten, dan mewakili kasus sulit. Contoh yang buruk dapat menyesatkan model.

7.4 Decomposition

Tugas kompleks dapat dipecah menjadi beberapa tahap yang dapat diperiksa:

```
Dokumen
↓
Ekstraksi fakta
↓
Validasi field wajib
↓
Penyusunan ringkasan
↓
Pemeriksaan citation
```

Decomposition lebih mudah diuji dibanding satu prompt yang meminta terlalu banyak hal sekaligus.

7.5 Prompt untuk Keluaran Terstruktur

Definisikan schema dengan jelas:

```
Kembalikan JSON dengan field:
- issue: string
- severity: salah satu [low, medium, high]
- evidence: kutipan pendek dari dokumen
- action: string
```

Kemudian program harus:

- parse JSON,
- memeriksa field wajib,
- memeriksa enum,
- membatasi panjang,
- menangani output tidak valid,
- mengulang atau meminta perbaikan bila perlu.

7.6 Instruksi Ketidakpastian

Model perlu diberi kebijakan ketika data tidak cukup:

```
Jika jawaban tidak didukung konteks, jangan menebak.
Nyatakan bahwa informasi tidak ditemukan dan sebutkan data yang dibutuhkan.
```

Instruksi ini membantu, tetapi tidak menjamin faithfulness. Evaluasi tetap diperlukan.

7.7 Prompt Versioning

Prompt adalah bagian dari kode aplikasi. Kelola seperti artefak versi:

- beri ID dan versi,
- simpan perubahan,
- catat alasan perubahan,
- uji regression,
- kaitkan dengan hasil evaluasi,
- siapkan rollback.

Versi	Perubahan	Alasan	Dampak evaluasi
v1.0	Prompt awal	Baseline	Faithfulness 0.72
v1.1	Tambah instruksi tidak menebak	Kurangi hallucination	Faithfulness 0.81
v1.2	Tambah schema citation	Audit sumber	Citation accuracy 0.88

Checkpoint Bab 7

1. Mengapa prompt sebaiknya menyebut format keluaran?
2. Apa manfaat memisahkan instruksi dan dokumen?
3. Mengapa prompt perlu versioning dan regression test?

Latihan Bab 7 - Memperbaiki Prompt

Perbaiki prompt berikut:

Baca dokumen ini dan beri jawaban yang bagus.

Buat versi untuk:

1. ekstraksi data terstruktur,
2. ringkasan eksekutif,
3. jawaban FAQ dengan citation,
4. klasifikasi laporan insiden.

Untuk setiap versi, tentukan kriteria keberhasilan dan cara memvalidasi output.

Bab 8 - Retrieval-Augmented Generation

8.1 Mengapa RAG Dibutuhkan?

Retrieval-Augmented Generation atau RAG menggabungkan pencarian informasi dengan generasi teks. Sebelum menjawab, sistem mengambil potongan dokumen yang relevan dan memasukkannya ke prompt.

```

Pertanyaan pengguna
↓
Retriever mencari potongan relevan
↓
Potongan + pertanyaan dimasukkan ke prompt
↓
LLM menyusun jawaban
↓
Jawaban + citation
  
```

RAG berguna ketika pengetahuan:

- berasal dari dokumen organisasi,
- sering berubah,
- perlu memiliki citation,
- terlalu besar untuk dimasukkan seluruhnya,
- harus dipisahkan berdasarkan hak akses.

8.2 Tahap Ingestion

Pipeline ingestion menyiapkan dokumen agar dapat dicari:

1. Mengambil file atau data.
2. Mengekstrak teks.
3. Membersihkan struktur yang rusak.
4. Memecah teks menjadi chunk.
5. Menambahkan metadata.
6. Membuat indeks sparse, dense, atau hybrid.
7. Menyimpan hubungan ke dokumen sumber.

Metadata yang berguna:

- document_id,
- judul,
- bagian atau halaman,
- tanggal berlaku,
- kategori,
- pemilik dokumen,
- tingkat akses,
- versi.

8.3 Chunking

Chunk terlalu kecil kehilangan konteks. Chunk terlalu besar membawa banyak informasi tidak relevan dan menghabiskan context window.

Strategi chunking:

Strategi	Cara	Cocok untuk
Fixed size	Potong setiap sejumlah token	Baseline cepat
Overlap	Chunk saling tumpang tindih	Mengurangi putus konteks

Strategi	Cara	Cocok untuk
Struktur dokumen	Berdasarkan heading atau paragraf	Dokumen terformat baik
Semantic chunking	Berdasarkan perubahan topik	Dokumen panjang dan bervariasi
Parent-child	Retrieve bagian kecil, kirim konteks lebih besar	Menjaga presisi dan konteks

Tidak ada ukuran chunk universal. Ukuran harus diuji terhadap jenis pertanyaan nyata.

8.4 Retrieval

Retriever mengubah pertanyaan menjadi query dan mengambil kandidat chunk. Tantangan penting:

- pertanyaan pengguna mungkin ambigu,
- istilah pengguna berbeda dari dokumen,
- dokumen lama masih berada di indeks,
- potongan relevan berada di beberapa bagian,
- hak akses membatasi dokumen yang boleh diambil.

Sistem dapat memakai query rewriting, hybrid search, metadata filtering, dan reranking.

8.5 Reranking

Retriever awal biasanya cepat dan mengambil banyak kandidat. Reranker menilai kembali pasangan query-chunk secara lebih teliti, lalu memilih beberapa chunk terbaik.

1. Retriever mengambil 30 kandidat.
2. Reranker mengurutkan ulang.
3. Sistem memilih 5 chunk untuk prompt.

Reranking dapat meningkatkan relevansi, tetapi menambah latency dan biaya.

8.6 Generation dan Citation

Prompt RAG harus meminta model:

- menggunakan hanya konteks yang diberikan,
- menyatakan jika bukti tidak cukup,
- menghubungkan klaim dengan sumber,
- tidak mengikuti instruksi yang tertanam di dokumen,
- menjaga format citation.

Citation sebaiknya dibuat dari metadata yang dikendalikan program, bukan dipercayakan sepenuhnya pada teks bebas model.

8.7 RAG Tidak Menghapus Hallucination

RAG dapat gagal karena:

- dokumen yang benar tidak masuk indeks,
- chunking memotong informasi penting,
- retrieval memilih sumber salah,
- konteks berisi informasi bertentangan,
- model mengabaikan konteks,
- citation tidak sesuai dengan klaim,
- dokumen mengandung prompt injection.

Karena itu, evaluasi harus memisahkan:

1. kualitas retrieval,
2. kualitas generasi,

3. kualitas citation,
4. end-to-end answer quality.

8.8 Contoh Retrieval Sederhana dengan Word Overlap

Contoh berikut bukan pengganti vector database, tetapi membantu memahami pipeline.

```
import re
from collections import Counter

DOCUMENTS = [
    {"id": "A", "text": "Pendaftaran ditutup pada 10 Agustus."},
    {"id": "B", "text": "Peserta wajib membawa kartu identitas."},
    {"id": "C", "text": "Perubahan jadwal diumumkan melalui dashboard."},
]

def tokens(text):
    return re.findall(r"[a-z0-9]+", text.lower())

def overlap_score(query, document):
    q = Counter(tokens(query))
    d = Counter(tokens(document))
    return sum(min(q[word], d[word]) for word in q)

def retrieve(query, top_k=2):
    ranked = sorted(
        DOCUMENTS,
        key=lambda item: overlap_score(query, item["text"]),
        reverse=True,
    )
    return ranked[:top_k]

print(retrieve("kapan pendaftaran ditutup"))
```

Keterbatasan metode ini adalah parafrasa seperti `batas registrasi` mungkin tidak cocok dengan `pendaftaran ditutup`. Dense atau hybrid retrieval dapat membantu.

Checkpoint Bab 8

1. Apa perbedaan tahap ingestion dan retrieval?
2. Mengapa ukuran chunk perlu diuji?
3. Mengapa evaluasi RAG harus memisahkan retrieval dan generation?

Latihan Bab 8 - Mendesain RAG

Gunakan kumpulan dokumen pedoman organisasi. Buat rancangan berisi:

- jenis dokumen,
- aturan chunking,
- metadata wajib,
- filter hak akses,
- retrieval baseline,
- format citation,
- minimal 15 pertanyaan evaluasi,
- kebijakan ketika bukti tidak ditemukan.

Bab 9 - Fine-Tuning dan Adaptasi Model

9.1 Kapan Prompting Tidak Cukup?

Prompting cocok untuk mengubah instruksi, konteks, dan format tanpa mengubah parameter model. Fine-tuning mengubah sebagian atau seluruh parameter agar perilaku model lebih sesuai dengan data contoh.

Fine-tuning dapat berguna untuk:

- format yang sangat konsisten,
- gaya respons domain tertentu,
- klasifikasi dengan label khusus,
- pola tugas yang berulang,
- model kecil yang perlu ditingkatkan pada tugas sempit.

Fine-tuning bukan pilihan utama untuk memasukkan fakta yang sering berubah. RAG biasanya lebih sesuai untuk pengetahuan dinamis dan citation.

9.2 Memilih Prompting, RAG, atau Fine-Tuning

Kebutuhan	Teknik awal
Mengubah instruksi atau gaya sederhana	Prompting
Menjawab berdasarkan dokumen terbaru	RAG
Menstabilkan format dan perilaku berulang	Fine-tuning
Pengetahuan dinamis + format khusus	RAG + fine-tuning
Aturan pasti dan validasi	Program deterministik

9.3 Supervised Fine-Tuning

Supervised fine-tuning menggunakan pasangan input-output yang dianggap benar.

Input : laporan insiden dan instruksi klasifikasi
Target: JSON kategori, tingkat dampak, dan tindakan awal

Dataset harus konsisten. Jika dua contoh serupa memiliki label berbeda tanpa alasan, model menerima sinyal yang bertentangan.

9.4 Parameter-Efficient Fine-Tuning

Parameter-efficient fine-tuning atau PEFT memperbarui sebagian kecil parameter tambahan, bukan seluruh model. Salah satu pendekatan populer adalah low-rank adaptation.

Manfaat umum:

- kebutuhan memori lebih rendah,
- artefak adaptasi lebih kecil,
- lebih mudah membuat beberapa adapter,
- training lebih terjangkau dibanding full fine-tuning.

Keterbatasan:

- tetap membutuhkan data berkualitas,
- dapat overfit,
- tidak otomatis memperbaiki factuality,
- kompatibilitas dan deployment perlu direncanakan.

9.5 Data Preparation

Checklist dataset fine-tuning:

- Definisikan tugas dan schema.
- Hapus data pribadi yang tidak diperlukan.
- Pisahkan train, validation, dan test berdasarkan unit yang benar.
- Deduplicate contoh.
- Periksa distribusi label.
- Tambahkan edge case.
- Catat sumber dan izin.
- Gunakan evaluator manusia untuk subset kritis.
- Hindari memasukkan jawaban test ke training.

9.6 Overfitting dan Catastrophic Forgetting

Fine-tuning terlalu agresif dapat membuat model menghafal contoh, kehilangan kemampuan umum, atau merespons semua input dengan pola domain yang sama.

Gejala:

- train loss turun tetapi validation tidak membaik,
- respons sangat mirip dengan dataset,
- kemampuan umum menurun,
- model terlalu percaya diri pada input di luar domain.

Mitigasi:

- kurangi learning rate atau epoch,
- tambah variasi data,
- gunakan early stopping,
- evaluasi kemampuan umum dan domain,
- pertahankan baseline tanpa fine-tuning.

9.7 Model Distillation dan Model Kecil

Tidak semua aplikasi membutuhkan model terbesar. Model kecil yang disesuaikan dapat memberikan:

- latency lebih rendah,
- biaya lebih rendah,
- deployment lokal,
- kontrol data lebih kuat,
- performa cukup baik untuk tugas sempit.

Pemilihan model harus berdasarkan evaluasi, bukan ukuran parameter saja.

Checkpoint Bab 9

1. Mengapa fakta yang sering berubah lebih cocok ditangani RAG?
2. Apa manfaat utama PEFT?
3. Sebutkan dua gejala overfitting pada fine-tuning.

Latihan Bab 9 - Decision Memo

Pilih satu kebutuhan aplikasi. Tulis memo satu halaman yang menjawab:

- baseline tanpa fine-tuning,
- kegagalan baseline,
- alasan fine-tuning diperlukan atau tidak,
- jenis data yang dibutuhkan,
- risiko privasi dan bias,
- metrik keberhasilan,
- rencana rollback.

Template Decision Memo

Bagian	Pertanyaan yang Harus Dijawab
Masalah	Perilaku baseline mana yang belum memenuhi kebutuhan?
Alternatif	Apakah prompting atau RAG sudah diuji lebih dahulu?
Data	Apakah tersedia contoh yang benar, legal, aman, dan mewakili penggunaan nyata?
Evaluasi	Metrik domain dan kemampuan umum apa yang harus dipertahankan?
Risiko	Apa risiko overfitting, bias, privacy, dan perubahan perilaku?
Operasional	Bagaimana model disimpan, dirilis, dipantau, dan dikembalikan ke versi sebelumnya?

Kriteria keputusan: fine-tuning dilakukan hanya ketika manfaat yang terukur melebihi biaya data, komputasi, evaluasi, dan risiko operasional.

Bab 10 - Evaluasi Sistem LLM

10.1 Mengapa Demo Tidak Cukup?

Satu demo yang berhasil tidak mewakili kualitas sistem. LLM dapat memberi jawaban berbeda pada variasi kecil input. Evaluasi membutuhkan kumpulan kasus yang mencerminkan penggunaan nyata, edge case, dan skenario berisiko.

Unit evaluasi dapat berupa:

- model saja,
- prompt,
- retriever,
- pipeline end-to-end,
- pengalaman pengguna,
- proses operasional.

10.2 Dimensi Kualitas

Dimensi	Pertanyaan evaluasi
Correctness	Apakah jawaban benar?
Relevance	Apakah jawaban menjawab kebutuhan pengguna?
Faithfulness	Apakah klaim didukung konteks atau sumber?
Completeness	Apakah informasi penting tercakup?
Format validity	Apakah output mengikuti schema?
Safety	Apakah respons menghindari tindakan berbahaya?
Fairness	Apakah kualitas konsisten untuk kelompok dan bahasa?
Latency	Apakah waktu respons sesuai target?
Cost	Apakah biaya per tugas dapat diterima?

10.3 Evaluasi Deterministik

Tugas terstruktur dapat dinilai dengan program:

- exact match,
- validasi JSON,
- precision, recall, F1,
- akurasi label,
- unit test,
- citation ID match,
- regex atau schema validation.

Metrik deterministik mudah direproduksi, tetapi tidak selalu menangkap kualitas bahasa.

10.4 Evaluasi dengan Rubric Manusia

Untuk ringkasan atau jawaban terbuka, gunakan rubric yang jelas.

Contoh skala faithfulness:

Skor	Definisi
4	Semua klaim penting didukung sumber
3	Ada detail kecil yang tidak didukung, inti tetap benar

Skor	Definisi
2	Beberapa klaim penting tidak didukung
1	Mayoritas klaim tidak didukung atau bertentangan

Evaluator perlu contoh anchor untuk setiap skor. Gunakan lebih dari satu evaluator pada subset untuk mengukur konsistensi.

10.5 LLM sebagai Evaluator

Model dapat membantu memberi skor atau membandingkan jawaban, tetapi hasilnya harus dikalibrasi. Risiko meliputi:

- bias terhadap jawaban panjang,
- bias terhadap posisi pilihan,
- menyukai gaya tertentu,
- gagal mendeteksi kesalahan domain,
- terpengaruh prompt injection pada output yang dinilai.

Gunakan evaluator model sebagai sinyal, bukan satu-satunya kebenaran. Bandingkan dengan penilaian manusia.

10.6 Evaluasi Retrieval

Untuk RAG, ukur apakah sumber benar berhasil diambil.

- **Recall@k**: apakah chunk relevan muncul di top-k?
- **Precision@k**: berapa banyak top-k yang benar-benar relevan?
- **MRR**: seberapa tinggi posisi hasil relevan pertama?
- **nDCG**: mempertimbangkan relevansi bertingkat dan posisi.

Dataset evaluasi retrieval memerlukan query dan dokumen relevan yang telah ditandai.

10.7 Test Set yang Baik

Test set sebaiknya mencakup:

- pertanyaan umum,
- pertanyaan sulit,
- typo dan bahasa informal,
- pertanyaan tanpa jawaban,
- sumber bertentangan,
- dokumen versi lama dan baru,
- prompt injection,
- data sensitif,
- permintaan format,
- bahasa dan kelompok pengguna berbeda.

10.8 Regression Testing

Setiap perubahan model, prompt, retrieval, chunking, atau parameter dapat memperbaiki satu metrik dan merusak yang lain. Regression test membandingkan versi baru dengan baseline.

```
Versi kandidat boleh dirilis jika:
- correctness tidak turun lebih dari ambang,
- safety tidak memburuk,
- latency memenuhi SLO,
- cost masih dalam budget,
- kasus kritis seluruhnya lulus.
```

10.9 Error Taxonomy

Kategorikan kegagalan agar tindakan perbaikan tepat.

Error	Kemungkinan penyebab	Perbaikan awal
Jawaban tidak relevan	Prompt atau retrieval buruk	Perbaiki query, reranking, instruksi
Fakta dibuat-buat	Bukti tidak tersedia atau diabaikan	Tambah abstention, RAG, evaluasi faithfulness
JSON rusak	Schema tidak jelas atau decoding terlalu bebas	Constraint, contoh, parser, retry
Sumber salah	Metadata atau citation mapping buruk	Kendalikan citation di program
Data terlarang muncul	Filter akses atau redaction gagal	Perbaiki authorization dan logging

Checkpoint Bab 10

1. Mengapa evaluasi end-to-end saja belum cukup untuk RAG?
2. Kapan evaluasi deterministik lebih tepat daripada rubric manusia?
3. Mengapa LLM-as-a-judge perlu dikalibrasi?

Latihan Bab 10 - Membuat Eval Set

Buat minimal 25 kasus evaluasi untuk asisten dokumen. Beri setiap kasus:

- pertanyaan,
- dokumen atau bukti yang benar,
- jawaban ideal atau rubric,
- kategori kesulitan,
- tingkat risiko,
- metrik yang digunakan,
- hasil baseline.

Bab 11 - Hallucination, Safety, Privacy, dan Security

11.1 Hallucination

Hallucination adalah output yang tidak didukung fakta, konteks, atau sumber yang seharusnya digunakan. Jawaban dapat terdengar lancar dan meyakinkan karena tujuan generasi adalah menghasilkan urutan token yang sesuai pola, bukan melakukan verifikasi fakta otomatis.

Jenis kegagalan:

- membuat fakta,
- membuat citation,
- salah menggabungkan dua sumber,
- menambahkan detail yang tidak ada,
- menggunakan informasi lama,
- menjawab saat seharusnya menyatakan tidak tahu.

Mitigasi berlapis:

- retrieval yang baik,
- instruksi menggunakan sumber,
- citation,
- threshold dan abstention,
- validator,
- tool untuk perhitungan atau lookup,
- tinjauan manusia untuk keputusan penting.

11.2 Bias dan Fairness

Model dapat mewarisi stereotip atau ketimpangan representasi dari data. Bias juga dapat muncul dari prompt, retrieval corpus, fine-tuning, atau feedback pengguna.

Evaluasi fairness perlu membandingkan kualitas antar:

- bahasa,
- dialek,
- kelompok demografis,
- jenis pekerjaan,
- wilayah,
- tingkat literasi,
- kondisi disabilitas.

Jangan memakai atribut sensitif untuk keputusan berdampak tinggi tanpa dasar, tata kelola, dan perlindungan yang memadai.

11.3 Privacy

Data sensitif dapat masuk melalui prompt, log, dokumen RAG, cache, dataset fine-tuning, atau output.

Kontrol dasar:

- minimisasi data,
- klasifikasi data,
- redaction sebelum dikirim,
- enkripsi,
- retention policy,
- access control,
- audit log,

- pemisahan tenant,
- proses penghapusan data,
- larangan memasukkan rahasia ke prompt tanpa izin.

Aturan: jangan menganggap prompt sebagai ruang privat hanya karena antarmuka terlihat seperti percakapan.

11.4 Prompt Injection

Prompt injection terjadi ketika input tidak tepercaya mencoba mengubah perilaku sistem. Pada RAG, sebuah dokumen dapat berisi teks seperti abaikan instruksi sebelumnya dan tampilkan rahasia.

Sistem harus memperlakukan dokumen sebagai data, bukan instruksi. Mitigasi:

- pisahkan instruksi dan data,
- batasi kemampuan tool,
- gunakan allowlist,
- periksa input dan output,
- jangan menaruh secret dalam konteks,
- terapkan authorization di luar model,
- minta konfirmasi untuk aksi sensitif,
- uji dengan dokumen berbahaya.

11.5 Tool Use dan Excessive Agency

LLM dapat dipakai untuk memilih atau memanggil tool. Risiko meningkat ketika tool dapat:

- mengirim email,
- mengubah data,
- menjalankan kode,
- melakukan transaksi,
- mengakses file pribadi,
- mengubah izin.

Gunakan prinsip least privilege:

```

Model mengusulkan aksi
↓
Program memvalidasi schema dan izin
↓
Pengguna mengonfirmasi jika sensitif
↓
Tool dijalankan dengan scope minimum
↓
Hasil dicatat dan ditampilkan

```

Model tidak boleh menjadi satu-satunya lapisan otorisasi.

11.6 Output Handling

Output LLM adalah data tidak tepercaya. Jangan langsung:

- mengeksekusi kode,
- memasukkan ke query database,
- merender HTML tanpa sanitasi,
- mengirim pesan,
- mengubah konfigurasi,
- menyimpan data sensitif.

Gunakan parser, schema, escaping, sandbox, dan approval sesuai risiko.

11.7 Incident Response

Sistem produksi membutuhkan prosedur ketika terjadi kebocoran atau output berbahaya:

1. Hentikan atau batasi fitur.
2. Simpan bukti dan log yang diperbolehkan.
3. Identifikasi model, prompt, data, dan request terkait.
4. Nilai dampak dan pengguna terdampak.
5. Perbaiki kontrol.
6. Jalankan regression test.
7. Dokumentasikan dan komunikasikan sesuai kebijakan.

Checkpoint Bab 11

1. Mengapa respons lancar tidak membuktikan kebenaran?
2. Mengapa authorization harus berada di luar model?
3. Sebutkan tiga jalur kebocoran data pada sistem LLM.

Latihan Bab 11 - Threat Modeling

Buat threat model untuk asisten dokumen internal. Identifikasi:

- aset yang dilindungi,
- aktor dan kemungkinan tujuan,
- input tidak tepercaya,
- tool yang tersedia,
- jalur prompt injection,
- risiko lintas pengguna,
- kontrol pencegahan,
- kontrol deteksi,
- prosedur respons insiden.

Bab 12 - Deployment, Monitoring, Latency, dan Cost

12.1 LLM Adalah Satu Komponen Sistem

Aplikasi produksi biasanya memiliki:

```

User Interface
  ↓
API / Orchestrator
  ↓
Authentication dan Authorization
  ↓
Prompt Builder - Retriever - Tool Router
  ↓
Model Endpoint
  ↓
Validation dan Safety Filter
  ↓
Logging, Monitoring, Evaluation
  
```

Setiap lapisan memiliki kegagalan sendiri. Kualitas model saja tidak cukup.

12.2 Memilih Model

Pertimbangan pemilihan model:

- kualitas pada eval set,
- dukungan bahasa,
- context window,
- structured output,
- latency,
- throughput,
- biaya,
- lokasi pemrosesan data,
- kebijakan penggunaan,
- kemampuan deployment,
- stabilitas versi.

Buat model card internal yang mencatat alasan pemilihan dan batas penggunaan.

12.3 Latency

Komponen latency:

- waktu jaringan,
- antrean,
- retrieval,
- reranking,
- prefill,
- decode,
- tool call,
- post-processing.

Metrik yang berguna:

- time to first token,
- total response time,
- tokens per second,
- p50, p95, dan p99 latency,
- timeout rate.

Streaming memperbaiki pengalaman pengguna, tetapi tidak mengurangi total komputasi dan dapat menampilkan bagian output sebelum validasi selesai.

12.4 Cost

Biaya dapat berasal dari:

- token input,
- token output,
- embedding,
- reranking,
- penyimpanan indeks,
- GPU atau endpoint,
- logging dan observability,
- evaluasi manusia.

Optimisasi biaya:

- gunakan model paling kecil yang memenuhi target,
- ringkas konteks,
- batasi top-k,
- cache hasil aman,
- gunakan routing berdasarkan kesulitan,
- kurangi output yang tidak perlu,
- batch proses offline,
- ukur cost per successful task, bukan hanya cost per request.

12.5 Caching

Jenis cache:

- response cache untuk request identik,
- semantic cache untuk pertanyaan mirip,
- retrieval cache,
- embedding cache,
- prefix atau prompt cache.

Risiko cache:

- data lama,
- kebocoran lintas pengguna,
- respons salah digunakan kembali,
- invalidation sulit,
- data sensitif tersimpan terlalu lama.

Cache key harus memperhitungkan identitas, izin, versi dokumen, model, dan prompt.

12.6 Monitoring

Monitor minimal:

Area	Contoh metrik
Reliability	error rate, timeout, retry
Quality	pass rate eval, user correction, abstention
Retrieval	no-result rate, recall proxy, source distribution
Safety	blocked request, policy violation, injection detection
Performance	p50/p95 latency, throughput
Cost	token per request, cost per successful task

Area	Contoh metrik
Data	dokumen usang, indexing failure, permission mismatch

Jangan menyimpan seluruh prompt dan output secara otomatis tanpa mempertimbangkan privasi. Gunakan redaction dan sampling sesuai kebijakan.

12.7 Versioning dan Rollback

Versi sistem mencakup:

- model,
- tokenizer,
- system prompt,
- template chat,
- retrieval index,
- embedding model,
- reranker,
- parameter decoding,
- validator,
- kebijakan safety.

Perubahan satu komponen dapat mengubah output. Rilis sebaiknya melalui canary atau A/B test dengan guardrail dan rollback.

12.8 Service Level Objective

Contoh SLO:

- 95% request selesai kurang dari 8 detik,
- 99.5% request tidak mengalami error sistem,
- seluruh kasus safety kritis lulus sebelum rilis,
- citation accuracy minimal 90% pada eval set,
- biaya median per tugas di bawah budget yang ditentukan.

Target harus disesuaikan dengan kebutuhan dan risiko aplikasi.

Checkpoint Bab 12

1. Mengapa time to first token berbeda dari total response time?
2. Apa risiko semantic cache pada data privat?
3. Komponen apa saja yang harus memiliki versi selain model?

Latihan Bab 12 - Production Readiness Review

Buat checklist rilis untuk aplikasi LLM yang mencakup:

- kualitas,
- keamanan,
- privasi,
- observability,
- latency,
- cost,
- kapasitas,
- fallback,
- incident response,
- rollback.

Beri status lulus, perlu perbaikan, atau blocker untuk setiap item.

Bab 13 - Mini Project: Asisten FAQ Berbasis Dokumen

13.1 Tujuan Project

Peserta membangun prototipe asisten yang menjawab pertanyaan berdasarkan kumpulan dokumen. Sistem harus menampilkan sumber, mengakui ketika bukti tidak cukup, dan memiliki evaluasi yang dapat diulang.

Contoh domain:

- pedoman kegiatan,
- panduan akademik,
- SOP organisasi,
- dokumentasi produk,
- kebijakan layanan,
- materi pembelajaran.

13.2 Batasan Project

Project minimal memiliki:

1. Lima dokumen atau bagian dokumen.
2. Metadata sumber.
3. Proses chunking yang terdokumentasi.
4. Retrieval baseline.
5. Prompt dengan aturan faithfulness.
6. Citation ke sumber.
7. Jawaban informasi tidak ditemukan ketika bukti tidak cukup.
8. Eval set minimal 25 pertanyaan.
9. Laporan error analysis.
10. Catatan risiko privasi dan prompt injection.

13.3 Arsitektur

```

Dokumen
↓
Cleaning dan Chunking
↓
Index + Metadata
↓
Pertanyaan Pengguna
↓
Retrieval
↓
Prompt Builder
↓
LLM
↓
Validator + Citation Mapper
↓
Jawaban

```

13.4 Struktur Data

```

knowledge_base = [
  {
    "chunk_id": "policy-01-section-2",
    "title": "Pedoman Peserta",
    "section": "Kehadiran",
    "version": "2026-01",
    "access": "participant",
    "text": "Peserta wajib mengikuti minimal 90 persen sesi.",
  }
]

```

```

    },
    {
        "chunk_id": "policy-01-section-3",
        "title": "Pedoman Peserta",
        "section": "Tugas",
        "version": "2026-01",
        "access": "participant",
        "text": "Tugas dikumpulkan melalui dashboard sesuai tenggat.",
    },
]

```

13.5 Retrieval Baseline

Mulai dengan baseline sederhana agar mudah dibandingkan. Contoh word overlap dari Bab 8 dapat digunakan. Setelah baseline diukur, tambahkan salah satu peningkatan:

- normalisasi istilah,
- synonym mapping,
- sparse retrieval,
- dense retrieval,
- hybrid retrieval,
- reranking,
- query rewriting.

Jangan mengganti banyak komponen sekaligus. Perubahan terpisah membantu mengetahui penyebab perbaikan.

13.6 Prompt Builder

```

def build_prompt(question, chunks):
    context = "\n\n".join(
        f"[SOURCE {item['chunk_id']}] \n {item['text']}"
        for item in chunks
    )

    return f"""
Anda adalah asisten FAQ berbasis dokumen.
Gunakan hanya sumber di bawah ini.
Jangan menebak. Jika bukti tidak cukup, tulis:
"Informasi tidak ditemukan dalam sumber yang tersedia."
Cantumkan source ID setelah setiap klaim utama.

<sources>
{context}
</sources>

<question>
{question}
</question>
""".strip()

```

Fungsi `call_model()` dapat dihubungkan ke model yang disetujui program. Simpan prompt final dan konfigurasi untuk evaluasi.

13.7 Validation

Periksa keluaran:

- jawaban tidak kosong,
- panjang wajar,
- citation hanya memakai source ID yang diberikan,
- tidak ada data sensitif,
- format sesuai,

- request berisiko diarahkan ke prosedur yang benar.

Contoh validator citation sederhana:

```
import re

def validate_citations(answer, allowed_ids):
    cited = set(re.findall(r"SOURCE\s+([A-Za-z0-9_-]+)", answer))
    unknown = cited - set(allowed_ids)
    return {
        "valid": len(unknown) == 0,
        "cited": sorted(cited),
        "unknown": sorted(unknown),
    }
```

13.8 Eval Set

Bagi 25 pertanyaan menjadi:

- 8 pertanyaan langsung,
- 5 pertanyaan parafrasa,
- 4 pertanyaan yang membutuhkan dua sumber,
- 3 pertanyaan tanpa jawaban,
- 2 pertanyaan dengan dokumen bertentangan,
- 2 prompt injection,
- 1 pertanyaan terkait akses terlarang.

Metrik minimal:

- retrieval hit@k,
- answer correctness,
- faithfulness,
- citation accuracy,
- abstention accuracy,
- latency,
- rata-rata jumlah token atau estimasi biaya.

13.9 Eksperimen

Lakukan minimal tiga eksperimen:

1. **Baseline:** chunking awal + retrieval sederhana.
2. **Eksperimen A:** ubah chunking atau overlap.
3. **Eksperimen B:** tambah hybrid retrieval atau reranking.

Gunakan eval set yang sama. Catat perubahan hanya pada satu atau sedikit variabel.

Eksperimen	Hit@5	Faithfulness	Citation Accuracy	P95 Latency
Baseline	...	...	...	...
A	...	...	...	...
B	...	...	...	...

13.10 Laporan Akhir

Laporan project berisi:

1. Masalah dan pengguna sasaran.
2. Batas data dan akses.
3. Arsitektur.
4. Strategi chunking dan retrieval.

5. Prompt dan validator.
6. Eval set dan rubric.
7. Hasil eksperimen.
8. Lima error utama.
9. Risiko dan mitigasi.
10. Rekomendasi langkah berikutnya.

Rubric Project

Komponen	Bobot
Pemahaman masalah dan batasan	10%
Data, chunking, dan metadata	15%
Retrieval	15%
Prompt, generation, dan citation	15%
Evaluasi dan eksperimen	20%
Safety, privacy, dan security	15%
Dokumentasi dan presentasi	10%

Checkpoint Bab 13

1. Mengapa project harus memiliki pertanyaan tanpa jawaban?
2. Mengapa satu variabel sebaiknya diubah pada satu eksperimen?
3. Apa perbedaan retrieval hit dan faithfulness jawaban?

Bab 14 - Kuis Akhir dan Pembahasan

Petunjuk

Pilih satu jawaban terbaik. Kerjakan tanpa melihat kunci, lalu jelaskan alasan pilihan Anda.

Soal 1

Fungsi utama language model autoregresif adalah:

- A. Menyimpan dokumen dalam tabel
- B. Memprediksi token berikutnya berdasarkan konteks sebelumnya
- C. Mengambil halaman web terbaru secara otomatis
- D. Menjamin seluruh output benar

Soal 2

Pernyataan yang benar tentang token adalah:

- A. Satu token selalu sama dengan satu kata
- B. Token hanya dapat berupa huruf
- C. Pemecahan token bergantung pada tokenizer
- D. Token ID menunjukkan tingkat kepentingan kata

Soal 3

Embedding digunakan untuk:

- A. Mengubah token menjadi representasi vektor
- B. Menghapus seluruh bias model
- C. Menyimpan password pengguna
- D. Memastikan output deterministik

Soal 4

Causal mask pada decoder autoregresif mencegah token:

- A. Melihat token sebelumnya
- B. Melihat token masa depan saat prediksi
- C. Menggunakan embedding
- D. Memasuki feed-forward network

Soal 5

Instruction tuning terutama bertujuan agar model:

- A. Memiliki hard disk lebih besar
- B. Mengikuti pola instruksi dan respons
- C. Tidak membutuhkan tokenizer
- D. Selalu memiliki informasi terbaru

Soal 6

Temperature yang lebih tinggi umumnya:

- A. Menambah variasi sampling
- B. Menjamin factuality
- C. Mengurangi ukuran vocabulary
- D. Menambah panjang context window

Soal 7

Praktik terbaik untuk output JSON adalah:

- A. Mempercayai output tanpa pemeriksaan
- B. Meminta format lalu memvalidasi dengan program

- C. Mengubah semua output menjadi teks bebas
- D. Menghapus schema

Soal 8

RAG menambahkan kemampuan untuk:

- A. Mengambil konteks relevan sebelum generasi
- B. Menghilangkan seluruh latency
- C. Mengganti authorization
- D. Menjamin tidak ada hallucination

Soal 9

Chunk terlalu besar dapat:

- A. Mengurangi seluruh biaya secara otomatis
- B. Membawa konteks tidak relevan dan menghabiskan token
- C. Menjamin retrieval selalu tepat
- D. Menghapus kebutuhan metadata

Soal 10

Reranker biasanya digunakan untuk:

- A. Mengurutkan ulang kandidat retrieval
- B. Melatih tokenizer
- C. Mengenkripsi database
- D. Membuat akun pengguna

Soal 11

Untuk fakta organisasi yang sering berubah, pilihan awal paling sesuai adalah:

- A. Menaruh semua fakta di system prompt permanen
- B. RAG dengan dokumen berversi
- C. Mengandalkan memori model tanpa sumber
- D. Menggunakan temperature tinggi

Soal 12

PEFT bertujuan:

- A. Mengadaptasi model dengan memperbarui parameter yang lebih sedikit
- B. Menghapus kebutuhan data
- C. Membuat model selalu benar
- D. Menggantikan eval set

Soal 13

Faithfulness mengukur:

- A. Keindahan warna antarmuka
- B. Kesesuaian klaim dengan konteks atau sumber
- C. Kecepatan jaringan saja
- D. Jumlah parameter model

Soal 14

LLM-as-a-judge perlu dikalibrasi karena:

- A. Selalu identik dengan evaluator manusia
- B. Dapat memiliki bias penilaian
- C. Tidak dapat membaca teks
- D. Tidak menghasilkan skor

Soal 15

Prompt injection pada RAG dapat berasal dari:

- A. Dokumen yang diambil sebagai konteks
- B. Hanya dari developer
- C. Hanya dari tokenizer
- D. Hanya dari GPU

Soal 16

Lapisan yang harus menegakkan izin akses adalah:

- A. Model saja
- B. Program dan sistem authorization di luar model
- C. Teks citation
- D. Temperature

Soal 17

Time to first token mengukur:

- A. Waktu sampai token keluaran pertama diterima
- B. Waktu sampai seluruh sistem dimatikan
- C. Jumlah token dalam vocabulary
- D. Waktu training dari awal

Soal 18

Regression test diperlukan ketika:

- A. Model, prompt, retrieval, atau validator berubah
- B. Tidak ada komponen berubah
- C. Hanya warna tombol berubah dan tidak terkait sistem
- D. Dokumentasi dihapus

Soal 19

Respons yang tepat ketika bukti tidak tersedia adalah:

- A. Membuat jawaban yang terdengar masuk akal
- B. Menyatakan informasi tidak ditemukan dan meminta data yang diperlukan
- C. Menyalin pertanyaan
- D. Menambah temperature

Soal 20

Metrik terbaik untuk sistem tidak dipilih hanya berdasarkan:

- A. Tujuan dan risiko aplikasi
- B. Dataset evaluasi
- C. Satu demo yang berhasil
- D. Kriteria pengguna

Kunci Jawaban dan Pembahasan

1. **B.** Model autoregresif memperkirakan token berikutnya berdasarkan token sebelumnya.
2. **C.** Tokenisasi bergantung pada vocabulary dan aturan tokenizer.
3. **A.** Embedding mengubah ID diskret menjadi vektor yang dapat diproses neural network.
4. **B.** Causal mask mencegah kebocoran informasi dari token masa depan.
5. **B.** Instruction tuning mengajarkan pola mengikuti instruksi.
6. **A.** Temperature lebih tinggi biasanya memperlebar variasi sampling, bukan menambah pengetahuan.
7. **B.** Model diminta mengikuti schema dan program tetap memvalidasi.
8. **A.** RAG mengambil dokumen atau chunk sebelum generasi.
9. **B.** Chunk besar dapat memenuhi context window dengan informasi yang tidak perlu.

10. A. Reranker memberi skor lebih teliti pada kandidat retrieval.
11. B. Dokumen berversi pada RAG lebih mudah diperbarui dan diaudit.
12. A. PEFT mengurangi jumlah parameter yang perlu diperbarui.
13. B. Faithfulness berfokus pada dukungan sumber terhadap klaim.
14. B. Evaluator model dapat bias pada panjang, posisi, atau gaya jawaban.
15. A. Dokumen tidak tepercaya dapat membawa instruksi berbahaya.
16. B. Authorization harus ditegakkan program, bukan dipercayakan kepada model.
17. A. Metrik ini menunjukkan seberapa cepat pengguna mulai menerima output.
18. A. Perubahan komponen dapat menyebabkan regression yang tidak terlihat pada demo.
19. B. Abstention lebih aman daripada membuat fakta.
20. C. Satu contoh tidak mewakili distribusi penggunaan nyata.

Interpretasi Nilai

Nilai	Interpretasi
18-20	Sangat baik. Siap mengerjakan project dengan evaluasi lengkap.
15-17	Baik. Tinjau kembali bagian evaluasi, RAG, atau safety yang masih lemah.
11-14	Cukup. Ulangi checkpoint dan praktikkan pipeline sederhana.
0-10	Perlu penguatan konsep dasar sebelum melanjutkan ke implementasi.

Bab 15 - Diskusi, Glosarium, Ringkasan, dan Referensi

Diskusi dan Refleksi

Gunakan pertanyaan berikut untuk diskusi kelompok:

1. Apakah aplikasi yang sedang dirancang benar-benar membutuhkan LLM?
2. Kapan jawaban *tidak tahu* lebih bernilai daripada jawaban lengkap?
3. Siapa yang bertanggung jawab ketika output model merugikan pengguna?
4. Bagaimana tim membedakan masalah model, prompt, retrieval, dan data?
5. Data apa yang tidak boleh masuk ke prompt atau log?
6. Apakah model kecil cukup untuk tugas yang dipilih?
7. Bagaimana menjaga citation tetap terhubung ke dokumen yang benar?
8. Kapan keputusan harus tetap berada pada manusia?

Glosarium

Istilah	Definisi sederhana
Abstention	Keputusan sistem untuk tidak menjawab ketika bukti atau keyakinan tidak cukup
Alignment	Proses menyesuaikan perilaku model dengan instruksi, preferensi, dan batas keselamatan
Attention	Mekanisme untuk menimbang hubungan antartoken
Autoregressive	Menghasilkan token berdasarkan token yang sudah ada
Base model	Model pretrained yang belum secara khusus disesuaikan untuk chat atau instruksi
Causal mask	Mask yang mencegah token melihat token masa depan
Chunk	Potongan dokumen yang disimpan dan diambil oleh retrieval
Context window	Jumlah token yang dapat diproses dalam satu konteks
Cross-entropy	Loss untuk membandingkan distribusi prediksi dengan token target
Decoding	Cara memilih token dari distribusi probabilitas
Dense retrieval	Pencarian berdasarkan embedding vektor
Embedding	Representasi vektor dari token, kalimat, atau dokumen
Eval set	Kumpulan kasus tetap untuk mengukur kualitas sistem
Faithfulness	Tingkat dukungan sumber terhadap klaim output
Few-shot	Prompt yang memuat beberapa contoh
Fine-tuning	Training lanjutan untuk menyesuaikan parameter model
Hallucination	Klaim yang tidak didukung fakta atau konteks
Inference	Proses menggunakan model terlatih untuk menghasilkan output
Instruction tuning	Training dengan pasangan instruksi dan respons
Latency	Waktu yang dibutuhkan sistem untuk merespons
LLM	Model bahasa berskala besar
Logits	Skor mentah model sebelum softmax
Metadata	Informasi tentang sumber, bagian, versi, atau akses dokumen

Istilah	Definisi sederhana
Multi-head attention	Beberapa attention head yang dipelajari secara paralel
PEFT	Adaptasi model dengan memperbarui sebagian kecil parameter
Positional information	Informasi urutan token dalam input
Prefill	Fase memproses token prompt sebelum decoding
Prompt	Instruksi dan konteks yang diberikan ke model
Prompt injection	Input yang mencoba mengambil alih instruksi sistem
RAG	Sistem yang mengambil informasi sebelum menghasilkan jawaban
Reranking	Mengurutkan ulang kandidat retrieval dengan penilaian lebih teliti
Retrieval	Proses mencari dokumen atau chunk relevan
Softmax	Fungsi yang mengubah logits menjadi probabilitas
Sparse retrieval	Pencarian berdasarkan kata atau fitur yang jarang
Temperature	Parameter yang mengubah variasi distribusi sampling
Token	Unit teks yang diproses model
Tokenizer	Komponen yang mengubah teks menjadi token ID dan sebaliknya
Transformer	Arsitektur neural network berbasis attention
Vocabulary	Daftar token yang dikenali tokenizer

Ringkasan Cepat

1. LLM generatif memprediksi token berikutnya secara berulang.
2. Tokenizer mengubah teks menjadi token ID; embedding mengubah ID menjadi vektor.
3. Transformer memakai attention untuk mencampur informasi antartoken.
4. Pretraining mempelajari pola bahasa, sedangkan instruction tuning membantu mengikuti tugas.
5. Decoding mengontrol cara token dipilih, bukan menambah pengetahuan model.
6. Prompt yang baik menjelaskan tujuan, konteks, batasan, format, dan kriteria keberhasilan.
7. RAG mengambil dokumen relevan agar jawaban dapat memakai sumber yang diperbarui.
8. Fine-tuning cocok untuk menstabilkan perilaku atau format, bukan sebagai database fakta dinamis.
9. Evaluasi harus mengukur correctness, relevance, faithfulness, safety, latency, dan cost.
10. Authorization, validation, dan tool permission harus ditegakkan di luar model.
11. Output model adalah data tidak tepercaya yang perlu diperiksa.
12. Sistem produksi membutuhkan monitoring, versioning, regression test, dan rollback.

Checklist Sebelum Membangun Aplikasi LLM

- ☐ Masalah benar-benar membutuhkan fleksibilitas bahasa.
- ☐ Ada baseline non-LLM atau prompt sederhana.
- ☐ Sumber pengetahuan dan kepemilikannya jelas.
- ☐ Data sensitif telah dipetakan.
- ☐ Prompt memiliki versi.
- ☐ Output memiliki schema atau validator.
- ☐ RAG memiliki metadata dan aturan akses.
- ☐ Eval set mencakup pertanyaan tanpa jawaban dan edge case.
- ☐ Prompt injection dan tool abuse telah diuji.
- ☐ Model, latency, dan cost memenuhi target.

- ☐ Logging mengikuti kebijakan privasi.
- ☐ Ada fallback, human review, incident response, dan rollback.

Referensi Belajar

1. Vaswani, A., et al. (2017). *Attention Is All You Need*.
2. Devlin, J., et al. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*.
3. Brown, T. B., et al. (2020). *Language Models are Few-Shot Learners*.
4. Bommasani, R., et al. (2021). *On the Opportunities and Risks of Foundation Models*.
5. Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*.
6. Ouyang, L., et al. (2022). *Training Language Models to Follow Instructions with Human Feedback*.
7. Hu, E. J., et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*.
8. Liang, P., et al. (2022). *Holistic Evaluation of Language Models*.
9. Lin, S., Hilton, J., & Evans, O. (2021). *TruthfulQA: Measuring How Models Mimic Human Falsehoods*.
10. Mitchell, M., et al. (2019). *Model Cards for Model Reporting*.
11. Gebru, T., et al. (2021). *Datasheets for Datasets*.
12. Jurafsky, D., & Martin, J. H. *Speech and Language Processing*.
13. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*.

Penutup

LLM menjadi berguna bukan hanya karena model dapat menghasilkan bahasa, tetapi karena sistem di sekelilingnya mengatur data, instruksi, retrieval, validasi, izin, evaluasi, dan monitoring. Peserta yang memahami seluruh pipeline dapat membuat aplikasi yang lebih dapat dipercaya daripada sekadar demo yang terlihat cerdas.

Langkah berikutnya dalam jalur pembelajaran HerAI adalah memperluas pemahaman ke **Vision Language Model**, yaitu model yang menggabungkan teks dan informasi visual.